

School Management App

Controller & Routes – Fejlesztői dokumentáció

Laravel 12 + Clean Code elvek | 2026

1. Mi a Controller és mi a feladata?

A Controller az a réteg amely a HTTP kérést fogadja és a JSON választ visszaadja. De a Controller maga semmi üzleti logikát nem végez – csak összeköti a többi réteget.

1.1 A Controller feladata – csak delegálás

```
class StudentController extends Controller
{
    // 1. Dependency Injection - Service megkapása
    public function __construct(
        private StudentService $studentService
    ) {}

    // 2. Kérés fogadása (Form Request validál automatikusan)
    // 3. Service hívása
    // 4. Resource visszaadása (JSON formázás)
    public function store(StoreStudentRequest $request): StudentResource
    {
        $student = $this->studentService->createStudent($request-
        >validated());
        return new StudentResource($student);
    }
}
```

Mi NEM a Controller feladata?

Validáció (ez a Form Request dolga), üzleti logika (ez a Service dolga), adatbázis lekérdezés (ez a Model dolga), JSON formázás (ez a Resource dolga). A Controller csak összeköti a rétegeket – semmi mást.

1.2 HTTP kérés útja a rendszerben

```
Frontend → GET /api/students/1
          ↓
Router   → StudentController@show($id)
```

```

↓
Form Req → authorize() + rules() (automatikusan)
↓
Controller → $this->studentService->getStudentById($id)
↓
Service → Student::with([...])->findOrFail($id)
↓
Model+DB → MySQL lekérdezés
↓
Service → Student objektum visszatérés
↓
Resource → new StudentResource($student) - JSON formázás
↓
Controller → return → JSON válasz
↓
Frontend → { "data": { "id": 1, "name": "..." } }

```

2. Dependency Injection a Controllerben

Minden Controller konstruktorában Dependency Injection-nel kapjuk meg a szükséges Service-eket. A Laravel IoC konténer automatikusan létrehozza és átadja a példányokat.

2.1 Egy Service – egyszerű eset

```

class StudentController extends Controller
{
    public function __construct(
        private StudentService $studentService
    ) {}
    // Laravel automatikusan létrehozza a StudentService példányt
}

```

2.2 Több Service – összetett eset

```

// AttendanceController - két Service-t injektál
class AttendanceController extends Controller
{
    public function __construct(
        private AttendanceService $attendanceService,
        private StudentService $studentService
    ) {}
    // A Laravel mindkét Service-t automatikusan átadja
}

```

```
// ExamController - két Service-t injektál
class ExamController extends Controller
{
    public function __construct(
        private ExamService $examService,
        private StudentService $studentService
    ) {}
}
```

Miért injektálunk több Service-t?

Néha egy Controller több domain adataival dolgozik. Az ExamController pl. vizsgákat kezel (ExamService), de a diák vizsgaeredményeihez a StudentService-re is szüksége van. Minden Service a saját felelősségét kezeli – a Controller csak összeköti őket.

3. A HTTP metódusok és visszatérési típusok

HTTP metódus	Controller metódus	Visszatér	HTTP státusz
GET (lista)	index()	AnonymousResourceCollection	200 OK
POST	store()	XxxResource	200 OK
GET (egy elem)	show()	XxxResource	200 OK
PUT/PATCH	update()	XxxResource	200 OK
DELETE	destroy()	JsonResponse	200 OK
POST (akció)	pay()/markAsRead()	XxxResource / JsonResponse	200 OK

Miért JsonResponse a destroy()-nál?

Törlés után nincs visszaadandó objektum – csak egy megerősítő üzenetet küldünk. A Resource Modellekhez való, nem szöveges üzenetekhez. Ezért response()->json(['message' => '...']) a helyes megoldás.

4. Route struktúra és grouping

A routes/api.php fájlban az összes API végpont definiálva van. A route-ok csoportokba vannak szervezve prefix, name és middleware szerint.

4.1 Miért middleware('auth:sanctum')?

```
Route::middleware('auth:sanctum')->group(function () {
    // Minden ide tartozó route-hoz be kell jelentkezni.
    // Ha nincs érvényes Sanctum token → 401 Unauthorized hiba
    Route::apiResource('students', StudentController::class);
    // ...
});
```

Mi az a Sanctum token?

A Laravel Sanctum egy API token autentikációs rendszer. Bejelentkezéskor a szerver egy token-t ad vissza, amit a frontend minden kéréshez csatol (Authorization: Bearer <token>). A middleware ellenőrzi ezt a token-t – ha érvényes, folytatódik a kérés, ha nem, 401-es hibát kap vissza.

4.2 Route::apiResource() – automatikus CRUD végpontok

```
// Ez az egy sor automatikusan létrehozza mind az 5 CRUD végpontot:
Route::apiResource('students', StudentController::class);

// Egyenértékű ezzel:
Route::get('/students', [StudentController::class, 'index']);
Route::post('/students', [StudentController::class, 'store']);
Route::get('/students/{id}', [StudentController::class, 'show']);
Route::put('/students/{id}', [StudentController::class, 'update']);
Route::delete('/students/{id}', [StudentController::class, 'destroy']);
```

4.3 Prefix és name grouping

```
// Prefix: minden route /api/attendances/-sal kezdődik
// Name: minden route neve attendances.-sal kezdődik
Route::prefix('attendances')->name('attendances.')->group(function () {
    Route::post('students', [AttendanceController::class,
        'storeStudents'])
        ->name('students.store');
    // Eredmény: POST /api/attendances/students
    // Route neve: attendances.students.store
});

// Miért kellene a route nevek?
// Mert bárhol a kódban hivatkozhatunk rájuk neve alapján:
// route('attendances.students.store') → '/api/attendances/students'
// Ha megváltozik az URL, a neve nem változik – a kód nem törik el.
```

4.4 Mikor NEM használunk apiResource()-t?

Nem minden modul illik a standard CRUD mintába. Ilyenkor kézzel definiáljuk a route-okat:

Modul	Miért nem apiResource()?
Attendance	Tömeges rögzítés, statisztikák – nem standard CRUD
Timetable	Osztályhoz és naphoz kötött lekérdezés, nem id alapján
Library	Két entitás (books + issues) egy prefix alatt
Transport	Extra műveletek: assign/remove diák
Notification	Az update() végpont nem kell – except(['update'])

5. Controller réteg – minden modul

5.1 StudentController – /api/students

Method	URI	Controller metódus	Leírás
GET	/api/students	index()	Az összes diák lapozással
POST	/api/students	store()	Új diák létrehozása
GET	/api/students/{id}	show()	Egy diák összes adatával
PUT	/api/students/{id}	update()	Diák adatainak frissítése
DELETE	/api/students/{id}	destroy()	Diák törlése (soft delete)

```
// Jellemző Controller metódus – a többi hasonló:
public function store(StoreStudentRequest $request): StudentResource
{
    $student = $this->studentService->createStudent($request-
>validated());
    return new StudentResource($student);
}
// 1. StoreStudentRequest → automatikus validáció és jogosultság
// 2. $request->validated() → csak a validált, biztonságos adat
// 3. Service hívás → üzleti logika (roll_number generálás stb.)
// 4. StudentResource → formázott JSON válasz
```

5.2 TeacherController – /api/teachers

Method	URI	Controller metódus	Leírás
GET	/api/teachers	index()	Az összes tanár lapozással

POST	/api/teachers	store()	Új tanár létrehozása
GET	/api/teachers/{id}	show()	Egy tanár összes adatával
PUT	/api/teachers/{id}	update()	Tanár adatainak frissítése
DELETE	/api/teachers/{id}	destroy()	Tanár törlése (soft delete)
GET	/api/teachers/{id}/schedule	schedule()	Napi beosztás (?day=monday)

A schedule() végpont – inline validáció

A schedule() metódusban a day paramétert inline validáljuk (\$request->validate()) mert ez csak egy query paraméter – nem érdemes külön Form Request osztályt létrehozni ilyen egyszerű esethez. Ez pragmatikus döntés: a Form Request akkor éri meg, ha összetett validáció kell.

5.3 SchoolClassController – /api/classes

Method	URI	Controller metódus	Leírás
GET	/api/classes	index()	Az összes osztály lapozással
POST	/api/classes	store()	Új osztály létrehozása
GET	/api/classes/{id}	show()	Egy osztály összes adatával
PUT	/api/classes/{id}	update()	Osztály adatainak frissítése
DELETE	/api/classes/{id}	destroy()	Osztály törlése (soft delete)
GET	/api/classes/{id}/students	students()	Osztály diákjainak listája

5.4 SubjectController – /api/subjects

Method	URI	Controller metódus	Leírás
GET	/api/subjects	index()	Az összes tantárgy
POST	/api/subjects	store()	Új tantárgy létrehozása
GET	/api/subjects/{id}	show()	Egy tantárgy összes adatával
PUT	/api/subjects/{id}	update()	Tantárgy frissítése
DELETE	/api/subjects/{id}	destroy()	Tantárgy törlése
GET	/api/subjects/teacher/{teacherId}	byTeacher()	Egy tanár tantárgyai

5.5 AttendanceController – /api/attendances

Az AttendanceController nem standard CRUD – tömeges rögzítés és statisztikák jellemzik. Két Service-t injektál: AttendanceService és StudentService.

Method	URI	Controller metódus	Leírás
POST	/api/attendances/students	storeStudents()	Diák jelenlét tömeges rögzítése
POST	/api/attendances/teachers	storeTeachers()	Tanár jelenlét tömeges rögzítése
GET	/api/attendances/class/{id}	byClass()	Osztály jelenlét egy napra
GET	/api/attendances/class/{id}/history	classHistory()	Osztály teljes előzménye
GET	/api/attendances/student/{id}	studentStats()	Diák statisztika

Miért plain JsonResponse a studentStats()-nál?

A statisztika nem egy Model objektum – hanem egy számított asszociatív tömb (total, present, absent, percentage stb.). A Resource osztályok Modellekhez valók. Számított adatokhoz response()->json(['data' => \$stats]) a helyes megoldás.

5.6 TimetableController – /api/timetables

Method	URI	Controller metódus	Leírás
GET	/api/timetables/class/{id}	byClass()	Egy osztály teljes órarendje
GET	/api/timetables/class/{id}/day	byClassAndDay()	Egy nap órarendje (?day=monday)
POST	/api/timetables	store()	Új órarend bejegyzés
PUT	/api/timetables/{id}	update()	Órarend bejegyzés frissítése
DELETE	/api/timetables/{id}	destroy()	Órarend bejegyzés törlése

5.7 ExamController – /api/exams

Az ExamController kezeli a vizsgák CRUD-ját, az eredmények tömeges rögzítését és a statisztikákat. Két Service-t injektál.

Method	URI	Controller metódus	Leírás
GET	/api/exams	index()	Az összes vizsga lapozással
POST	/api/exams	store()	Új vizsga létrehozása
GET	/api/exams/{id}	show()	Egy vizsga összes adatával

PUT	/api/exams/{id}	update()	Vizsga frissítése
DELETE	/api/exams/{id}	destroy()	Vizsga törlése
POST	/api/exams/{id}/results	storeResults()	Eredmények tömeges rögzítése
GET	/api/exams/{id}/statistics	statistics()	Vizsga statisztikái
GET	/api/exams/student/{id}/results	studentResults()	Diák összes eredménye

A storeResults() – grade és status nem a frontendről jön

A vizsgaeredmények rögzítésekor a frontend csak a marks_obtained-et küldi. A grade (A+, A, B+ stb.) és a status (pass/fail) automatikusan kiszámítódik az ExamService-ben a calculateGrade() private metódussal. Ez az üzleti logika védelme – a felhasználó nem tudja manipulálni az osztályzatot.

5.8 FeeController – /api/fees

Method	URI	Controller metódus	Leírás
GET	/api/fees	index()	Az összes díj lapozással
POST	/api/fees	store()	Új díj létrehozása
GET	/api/fees/{id}	show()	Egy díj adatai
PUT	/api/fees/{id}	update()	Díj adatainak frissítése
DELETE	/api/fees/{id}	destroy()	Díj törlése
GET	/api/fees/student/{id}	byStudent()	Diák összes díja
GET	/api/fees/overdue/list	overdue()	Összes lejárt díj
POST	/api/fees/{id}/pay	pay()	Díj befizetésének rögzítése

Miért külön pay() végpont?

A befizetés nem egyszerű update – a FeeService.markAsPaid() automatikusan beállítja a státuszt, a paid_date-et és generálja a receipt_number-t. A POST /fees/{id}/pay szemantikailag is egyértelműbb mint PUT /fees/{id} {status: paid}. Az akció-alapú végpontok (pay, assign, return) jobban kifejezik a szándékot.

5.9 LibraryController – /api/library

A könyvtár két entitást kezel egy prefix alatt: könyvek (/library/books) és kölcsönzések (/library/issues).

Method	URI	Controller metódus	Leírás
--------	-----	--------------------	--------

GET	/api/library/books	index()	Könyvek (?available=true)
POST	/api/library/books	store()	Új könyv hozzáadása
GET	/api/library/books/{id}	show()	Egy könyv adatai
PUT	/api/library/books/{id}	update()	Könyv frissítése
DELETE	/api/library/books/{id}	destroy()	Könyv törlése
POST	/api/library/books/{id}/issue	issue()	Könyv kölcsönzése
POST	/api/library/issues/{id}/return	returnBook()	Könyv visszahozása
GET	/api/library/issues/active	activeIssues()	Aktív kölcsönzések
GET	/api/library/issues/overdue	overdueIssues()	Lejárt kölcsönzések

A ?available=true query paraméter

A `$request->boolean('available')` automatikusan true/false értéké alakítja a query paramétert. Ha `?available=true` → csak az elérhető könyveket adja vissza. Ha nincs `?available` paraméter → az összes könyvet. Ez egy egyszerű, rugalmas szűrési megoldás.

5.10 TransportController – /api/transport

Method	URI	Controller metódus	Leírás
GET	/api/transport	index()	Az összes járat
POST	/api/transport	store()	Új járat létrehozása
GET	/api/transport/{id}	show()	Egy járat adatai
PUT	/api/transport/{id}	update()	Járat frissítése
DELETE	/api/transport/{id}	destroy()	Járat törlése
POST	/api/transport/{id}/assign	assignStudent()	Diák hozzárendelése
DELETE	/api/transport/{id}/remove/{sid}	removeStudent()	Diák eltávolítása

5.11 NotificationController – /api/notification

Method	URI	Controller metódus	Leírás
GET	/api/notification	index()	Az összes értesítés
POST	/api/notification	store()	Új értesítés létrehozása
GET	/api/notification/{id}	show()	Egy értesítés adatai
DELETE	/api/notification/{id}	destroy()	Értesítés törlése
POST	/api/notification/{id}/read	markAsRead()	Olvasottnak jelölés

GET	/api/notifications/meta/unread-count	unreadCount()	Olvasatlan szám
GET	/api/notifications/meta/user	userNotifications()	Saját értesítések

Miért nincs update() az értesítésekénél?

Az értesítéseket nem módosítjuk – csak létrehozunk, olvassuk, töröljük és olvasottnak jelöljük. A `Route::apiResource()->except(['update'])` pontosan ezt fejezi ki: az összes standard CRUD végpont megvan, kivéve az update-et. Ez clean code elv – csak azt tesszük elérhetővé amire ténylegesen szükség van.

auth()->user() a Controllerben

A `store()` és `markAsRead()` metódusokban az `auth()->user()` a Sanctum middleware által azonosított bejelentkezett felhasználót adja vissza. Ezt adjuk át a Service-nek (`created_by`, `user_id`), hogy tudjuk ki hozta létre az értesítést és ki jelölte olvasottnak.

6. Az összes API végpont összefoglalója

Modul	Végpontok száma	Extra műveletek
StudentController	5	–
TeacherController	6	<code>schedule()</code>
SchoolClassController	6	<code>students()</code>
SubjectController	6	<code>byTeacher()</code>
AttendanceController	5	<code>storeStudents</code> , <code>storeTeachers</code> , <code>byClass</code> , <code>history</code> , <code>stats</code>
TimetableController	5	<code>byClass</code> , <code>byClassAndDay</code>
ExamController	8	<code>storeResults</code> , <code>statistics</code> , <code>studentResults</code>
FeeController	8	<code>byStudent</code> , <code>overdue</code> , <code>pay</code>
LibraryController	9	<code>issue</code> , <code>returnBook</code> , <code>activeIssues</code> , <code>overdueIssues</code>
TransportController	7	<code>assignStudent</code> , <code>removeStudent</code>
NotificationController	7	<code>markAsRead</code> , <code>unreadCount</code> , <code>userNotifications</code>
ÖSSZESEN	72	–

7. Controller mappa struktúra

```

app/
├── Http/
│   └── Controllers/
│       └── Api/
│           ├── StudentController.php
│           ├── TeacherController.php
│           ├── SchoolClassController.php
│           ├── SubjectController.php
│           ├── AttendanceController.php
│           ├── TimetableController.php
│           ├── ExamController.php
│           ├── FeeController.php
│           ├── LibraryController.php
│           ├── TransportController.php
│           └── NotificationController.php
└── routes/
    └── api.php ← összes API végpont, auth:sanctum middleware alatt

```

8. Clean Code elvek a Controller és Route rétegben

Elv	Hogyan valósítottuk meg
Single Responsibility	Controller csak delegál – validáció, logika, formázás mind külön rétegben
Dependency Injection	Konstruktorban kapja a Service-eket – nem maga hozza létre
Route grouping	Prefix + name + middleware csoportok – ismétlés nélkül
apiResource()	5 CRUD végpont egy sorban – DRY (Don't Repeat Yourself)
except(['update'])	Csak szükséges végpontok – felesleges route-ok nincsenek
Inline validáció	Egyszerű query paraméterekhez – Form Request csak ha összetett
Akció végpontok	pay(), markAsRead(), issue() – szemantikus, egyértelmű URL-ek
auth()->user()	Sanctum middleware azonosítja – Controller csak használja
Route nevek	Minden route-nak neve van – URL változásakor a kód nem törlik
namespace Api\	Elkülönített API Controller namespace – tiszta mappastruktúra

– School Management App fejlesztési dokumentáció –